LECTURE : 10

Revision Quiz: Closures, Scope & “this”

Questions — spot the closure, trace lexical scope through var / let / const, and call out “this” in regular vs. arrow functions.

Assume: browser context, non-strict mode, ES6+

FORMAT

How this revision works

Q1–Q2	**Identify the Closure**	Spot which function is a closure and predict what it remembers.
Q3–Q6	**Lexical Scope: var vs let vs const**	Trace scope and hoisting through loops, blocks, and re-declarations.
Q7–Q10	**“this”: Regular vs. Arrow**	Decide what “this” resolves to in each function type and context.

1

Identify the Closure

Which function keeps a live link to a variable it no longer “lives” next to?

What does this print?

```
function makeCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const counter = makeCounter();
console.log(counter());
console.log(counter());
```

A `1, 1`

B `1, 2`

C `undefined, undefined`

D `ReferenceError`

ANSWER

B) 1, 2

A	B ✓	C	D
`1, 1`	`1, 2`	`undefined, undefined`	`ReferenceError`

- The returned function is a closure over count — it keeps a live reference to the variable from makeCounter()'s scope.
- Each call to counter() reuses that same count, it doesn’t reset to 0, because makeCounter() only ran once.
- First call: count becomes 1 and returns 1. Second call: count becomes 2 and returns 2.

outer is set to null before fn() runs. What is logged?

```
function outer() {
  var msg = "hello";
  return function inner() {
    console.log(msg);
  };
}

let fn = outer();
outer = null;
fn();
```

?

Predict exactly what gets logged — and note whether nulling out "outer" changes anything.

ANSWER

```
"hello"
```

- inner() forms a closure over msg the moment it’s created inside outer() — that link doesn’t depend on the outer variable still pointing to the function.
- Reassigning outer = null only removes one reference to the outer() function object; it has no effect on fn, which already holds its own closure.
- As long as something (fn) still references inner, JavaScript keeps msg alive in memory — that’s the whole point of a closure.

2

Lexical Scope: var, let, const

Same loop, three declarations — three very different outcomes.

QUESTION 3 · LEXICAL SCOPE

What does console.log(x) print inside b()?

```
let x = 10;

function a() {
  let x = 20;
  function b() {
    console.log(x);
  }
  b();
}

a();
```

A 10

B 20

C undefined

D ReferenceError

ANSWER

B) 20

A	B ✓	C	D
10	20	undefined	ReferenceError

- b() is physically written inside a(), so its scope chain is: b → a → global.
- JavaScript looks for x starting in b()'s own scope (not found), then the next scope out — a() — where x = 20 is found first.
- The global x = 10 is never reached, because lexical scope stops climbing as soon as it finds a match — it depends on where b() is written, not that a() was called from the global scope.

QUESTION 4 · CONST

Which line throws an error?

```
const arr = [1, 2, 3];
arr.push(4);       // line A
arr[0] = 99;       // line B
arr = [5, 6];      // line C
```

A Line A

B Line B

C Line C

D None of them

ANSWER

C) `arr = [5, 6];`

A	B	C ✓	D
Line A	Line B	Line C	None of them

- const only locks the binding — the variable name can never be pointed at a new value or object.
- It says nothing about the contents of an object or array: push() and index assignment both mutate the existing array in place, which is completely legal.
- Line C tries to reassign arr to a brand-new array, which violates the const binding — that's the TypeError.

3

“this”: Regular vs. Arrow

Same object, same property — does the function type change what “this” means?

greetFn is called on its own, detached from user. What logs?

```
const user = {
  name: "Maya",
  greet: function () {
    console.log(this.name);
  }
};

const greetFn = user.greet;
greetFn();
```

A "Maya"

B undefined

C ReferenceError

D "user"

ANSWER

B) undefined

A	B ✓	C	D
`"Maya"`	`undefined`	`ReferenceError`	`"user"`

- A regular function's “this” is decided entirely by the call site — not by where the function was originally defined or attached.
- greetFn() is a plain, detached call (no object before the dot), so “this” falls back to the default binding: the Window object in a browser.
- Window has no name property, so this.name reads undefined — the link to user was lost the moment greet was copied into greetFn.

Two methods, same object. What does each one log?

```
const user = {
  name: "Maya",
  regular: function () { console.log(this.name); },
  arrow: () => { console.log(this.name); }
};

user.regular();
user.arrow();
```

?

Predict both outputs, in order — one for user.regular() and one for user.arrow().

ANSWER

"Maya" then undefined

- regular is a normal function, so calling it as user.regular() implicitly binds “this” to user — this.name reads “Maya”.
- arrow is an arrow function, so it has no “this” of its own — it lexically inherits “this” from where it was written: the top level, where “this” is Window.
- Calling it as user.arrow() makes no difference — arrow functions always ignore the call site.

QUESTION 7 · THIS · NESTED ARROW

An arrow function is defined inside a regular method. What logs?

```
const user = {
  name: "Maya",
  greet: function () {
    const inner = () => {
      console.log(this.name);
    };
    inner();
  }
};

user.greet();
```

A "Maya"

B undefined

C ReferenceError

D "Window"

ANSWER

A) "Maya"

A ✓	B	C	D
"Maya"	undefined	ReferenceError	"Window"

- greet is a regular function; calling user.greet() implicitly binds its “this” to user.
- inner is an arrow function defined lexically inside greet, so it inherits greet's “this” — which is user, not Window.
- That's why this.name inside inner still resolves to “Maya”, even though inner() is invoked as a plain call.

WRAP-UP

Three rules to carry forward

1 **Closures**

A function keeps a live link to the variables of the scope it was created in — even after that scope has finished running.

2 **Lexical Scope**

var is function-scoped and shared; let/const are block-scoped, giving each loop iteration (or block) its own binding.

3 **“this”**

Regular functions get “this” from the call site. Arrow functions never do — they inherit it from their enclosing scope, permanently.



How did you score?

Revisit any question you missed — the explanation slide is right after it.

```
console.log("Lecture 10 complete");
```